

HPC, Laboratoire n°5 : Profiling

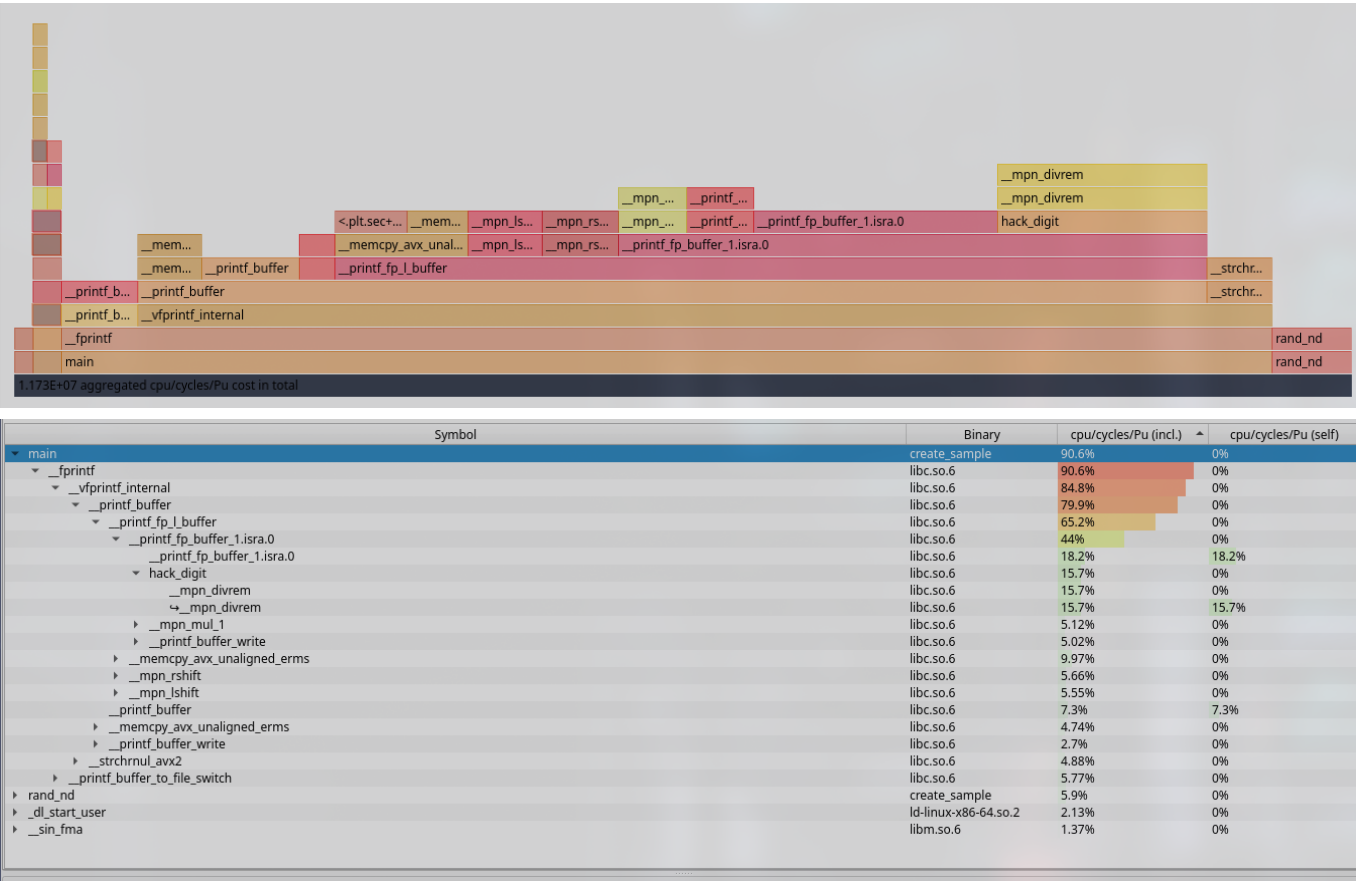
Student : Emily Baquerizo

Note

J'ai eu des soucis lors de l'installation de Valgrind (et donc cachegrind et callgrind) sur ma machine. Ainsi, je n'ai pas pu utiliser cet outil pour ce laboratoire donc mes analyses sont basées uniquement sur l'outil `perf` et `hotspot`

5.6 Analyse

Le profiling de l'exécutable `create_sample` montre que la grande majorité du temps d'exécution (environ 90%) est passée dans la fonction d'affichage `fprintf` et les fonctions internes associées à la gestion du buffer d'écriture.



En dehors de la phase d'écriture, il n'y a pas de fonctions vraiment coûteuses, ce qui semble logique vu que la génération des données est assez simple.

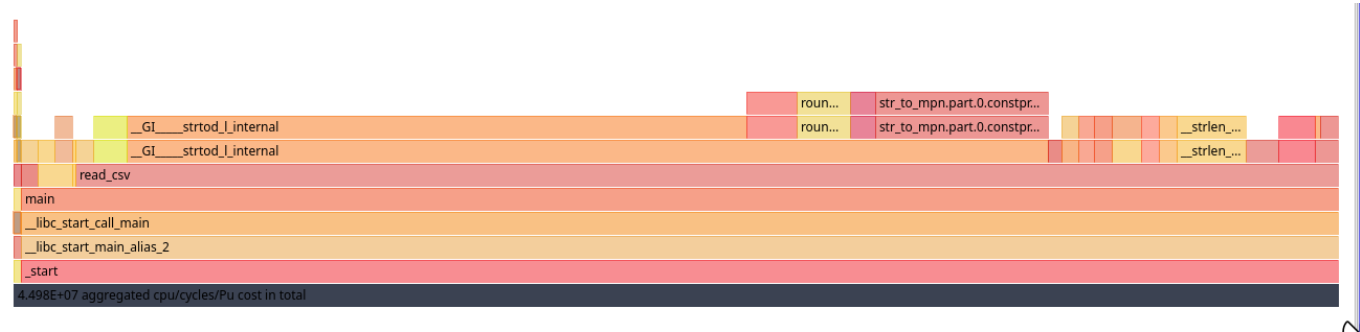
Le point problématique dans nos performances est le temps passé en écriture de fichier, et non la génération des valeurs ou le calcul en lui-même.

5.7 Profiling

Analyse performance

Le profiling du programme du laboratoire 01 montre que la quasi-totalité du temps d'exécution est passée dans la fonction `read_csv`, principalement lors de `strtod`. Ces résultats ont été conclu avec `perf` et `hotspot`.

Symbol	Binary	cpu/cycles/Pu (incl.)	cpu/cycles/Pu (self)
▼ _start	ecg_dealination	99.4%	0%
▸ _libc_start_main_alias_2	libc.so.6	99.4%	0%
↳ _libc_start_call_main	libc.so.6	99.4%	0%
▼ ↪main	ecg_dealination	99.4%	0%
▸ read_csv	ecg_dealination	95.3%	0%
▸ _GI____strtod_l_internal	libc.so.6	69.5%	0%
▸ _strlen_avx2	libc.so.6	5.25%	0%
▸ skip_spaces	ecg_dealination	2.73%	0%
▸ ??	libc.so.6	2.51%	0%
▸ read_csv	ecg_dealination	2.47%	2.47%
▸ __memmove_chk_avx_unaligned_erms	libc.so.6	2.21%	0%
▸ str_to_mpn.part.0.constprop.0	libc.so.6	1.72%	0%
▸ __mpn_lshift	libc.so.6	1.33%	0%
▸ __memcpy_avx_unaligned_erms	libc.so.6	1.33%	0%
▸ ??		1.33%	1.33%
▸ __mpn_construct_double	libc.so.6	1.32%	0%
▸ _ctype_b_loc	libc.so.6	1.25%	0%
▸ __memchr_avx2	libc.so.6	1.23%	0%
▸ _GI__errno_location	libc.so.6	1.03%	1.03%
▸ ecg_analyze	ecg_dealination	2.62%	0%
▸ _GI__errno_location	libc.so.6	1.28%	0%
▸ fgetc	libc.so.6	0.279%	0%
▸ _dl_start_user	ld-linux-x86-64.so.2	0.556%	0%



Amélioration possible

Une amélioration simple serait de remplacer `strtod` par une fonction de conversion optimisée ou d'utiliser une bibliothèque de parsing CSV plus rapide. On pourrait aussi envisager de paralléliser la lecture du fichier si le format s'y prête.